

CivicPulse — Source Code Documentation

Technical reference for the whole system: how it is put together, why it is put together that way, and where each rule actually lives.

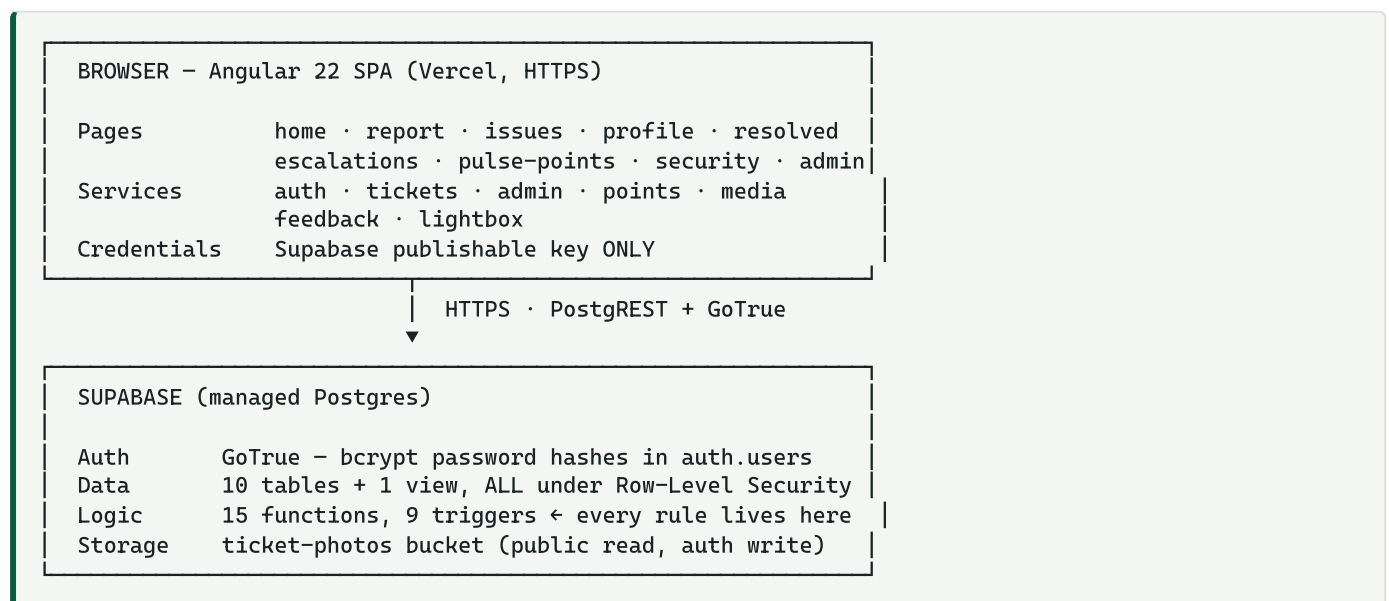
- **Live:** <https://civicpulse-black.vercel.app>
 - **Repository:** <https://github.com/lkjotsingh2710/CivicPulse>
 - **Scale:** 76 tracked files · ~9,000 lines of TypeScript · ~1,250 lines of SQL
-

1. What the system does

A citizen photographs a problem on their campus or in their city — a pothole, a dead streetlight, uncollected waste, a water leak. The photo must be taken live, through the app, and is geotagged at the moment of capture. The report goes to a ward desk, which works it through a status pipeline and uploads proof when it is fixed. Other people who are affected by the same problem back it, pushing it up the desk's queue. When the desk accepts a campus report, the reporter earns Pulse Points, redeemable for vouchers.

The design problem underneath all of that is **trust**. Points are worth money, so the system has to assume the person using it may be actively trying to cheat it. That single assumption explains most of the architecture below.

2. Architecture



There is **no application server**. The browser talks to Postgres directly through PostgREST, and the database decides what each caller may see and do. That is only safe because of the rule in the next section.

Tech stack

Layer	Choice	Why
Frontend	Angular 22, standalone components, signals	Zoneless-friendly reactivity; every page lazy-loaded
Styling	Plain SCSS + CSS custom properties	No framework; one token file drives the whole palette
Backend	Supabase (Postgres 15 + PostgREST + GoTrue)	Free tier; RLS makes a serverless architecture safe
Storage	Supabase Storage (Cloudinary optional)	Falls back automatically when Cloudinary is unset
Maps	MapLibre GL + hand-built SVG backdrop	Open source, no API key, no per-load billing
Tests	Vitest	13 tests over identity normalisation and CSV export
Hosting	Vercel Hobby	Free HTTPS — which the camera API requires

Every dependency is free. No paid API is in the critical path.

3. The security model — read this first

The browser is not trusted, and the code in it is not a security boundary.

The only credential that ships to the browser is the Supabase *publishable* key, which is public by design. Anyone can open DevTools, read it, and issue any request they like. The system survives that because **every rule that matters is a Postgres policy, trigger or function**, not a check in TypeScript.

Three mechanisms do the work:

Row-Level Security decides which rows a caller can see or change. A citizen matches on `user_phone = current_phone()`; an admin matches on `is_admin()`. Both read from the JWT, which the client cannot forge.

Triggers overwrite rather than validate. RLS grants access to a *row*, not to a *column* — so a citizen with permission to edit their own ticket could otherwise PATCH it straight to `Resolved`. `guard_workflow_columns` simply copies the old values back over `status`, `resolution_image_url`, `rejection_reason` and friends for non-admins. There is no rule to get subtly wrong; the column is incapable of holding an unauthorised value.

Privileged writes go through SECURITY DEFINER functions. Points are awarded by a trigger and spent by `redeem_reward()`. There is no INSERT policy on `pulse_points` at all, for anyone.

Verified, not assumed

These were tested against the live database with only the publishable key:

Attack	Result
<code>INSERT</code> 9999 points into <code>pulse_points</code>	<code>new row violates row-level security policy</code>
<code>redeem_reward()</code> while signed out	<code>Sign in before redeeming a reward</code>
Read another citizen's ledger	0 rows
Read the voucher-code pool	0 rows
List every photo in the storage bucket	blocked (migration 0006)

The bucket-listing case is worth noting: the original policy in `0001` granted `SELECT` on `storage.objects` to the `public` role, which permits the *list* operation, not just fetching a file you already have a link to. Anyone with the publishable key could enumerate every report photo ever uploaded. Migration `0006` drops it — public buckets serve files through the bucket's own `public` flag and do not need that policy.

4. Identity and authentication

CivicPulse has **no OTP and no SMS cost**. Citizens register with name, phone and password.

Supabase Auth is an email/password provider, so the client synthesises a stable address from the phone digits:

```
7428892131 → 7428892131@citizens.civicpulse.app
```

The domain is deliberately unregistered and cannot receive mail. The real identity lives in user metadata, and that is what every policy compares:

```
create function public.current_phone() returns text language sql stable as $$
  select nullif(auth.jwt() -> 'user_metadata' ->> 'phone', '');
$$;
```

Passwords are never stored by this application. They go straight to GoTrue, which bcrypt-hashes them into `auth.users.encrypted_password`. No table in `public` has a password column, and `auth.users` is unreachable from the browser. Even with the service-role key you would see only a one-way hash.

Phone normalisation is identity

`normalisePhone` is the single most consequential function in the frontend. Its output becomes the auth email, the JWT claim, `grievance_tickets.user_phone`, and `admin_users.phone` — so a change to it is an identity change that would orphan every existing user from their own reports.

It canonicalises the Indian country and trunk prefixes, because `+91 74288 92131`, `07428892131` and `7428892131` are one person writing one number but three different strings:

```
+91 74288 92131 → 7428892131
0917428892131 → 7428892131
07428892131 → 7428892131
+1 415 555 0199 → 14155550199 (not Indian – left exactly as typed)
```

Five unit tests in `auth.service.spec.ts` exist as a tripwire so nobody changes this casually.

Admin access

There is no separate admin credential. `/admin` requires a row in `admin_users` matching your phone — you register as a normal citizen and get promoted by `0005_grant_admin.sql`. The route guard is a convenience that keeps citizens off a screen of empty tables; the real boundary is that every admin query returns nothing without that row.

Every admin sees every ward. `ward_location` on `admin_users` is a header label, not a permission scope.

5. Data model

Ten tables and one view, all with RLS enabled.

Table	Purpose	Who can read	Who can write
<code>grievance_tickets</code>	The reports	own rows / admin all	own insert; workflow columns admin-only
<code>admin_users</code>	Who runs a desk	own row only	nobody from the client
<code>profiles</code>	Name + phone mirrored from <code>auth.users</code>	own / admin	trigger only
<code>platform_feedback</code>	Complaints about the app itself	admin only	anyone, signed in or not
<code>ticket_upvotes</code>	"I have this problem too"	own / admin	own insert + delete
<code>pulse_points</code>	Append-only points ledger	own / admin	nobody — trigger + function only
<code>rewards</code>	Reward catalogue	everyone	nobody from the client
<code>reward_codes</code>	Voucher code pool	nobody — no policy exists	nobody
<code>redemptions</code>	Claimed rewards	own / admin	function only
<code>campuses</code>	Which wards earn points	everyone	nobody
<code>public_tickets</code> (view)	The public feed	everyone incl. signed out	n/a

`grievance_tickets` — the core

```
id · ticket_number (CP-2026-00001) · created_at · updated_at
user_phone · user_name
category · description · urgency · ward_location
latitude · longitude · image_url
status      Submitted → In Progress → Resolved
            \ Rejected (terminal)
rejection_reason · resolution_image_url
upvote_count · department_email · last_escalated_at
```

`ticket_number` is assigned by a trigger from a sequence, so it is never chosen by the client.

`public_tickets` — the privacy boundary

The public feed is a view, and **its column list is the privacy policy**:

```
create view public.public_tickets as
select id, ticket_number, created_at, category, description, urgency,
       latitude, longitude, ward_location, image_url, status,
       upvote_count, user_name           -- name published
from public.grievance_tickets           -- user_phone deliberately absent
where t.status <> 'Rejected';            -- don't publish someone's mistake
```

It is intentionally **not** `security_invoker`, so it runs as its owner and bypasses the row policies on the underlying table. That is the only way to show one citizen another citizen's report without opening up the table itself. Every column that leaves has been chosen by hand.

6. Feature walkthrough

6.1 Filing a report

Live camera only. `camera-capture.ts` uses `getUserMedia` — **there is no file input for citizens**, so a saved meme or a downloaded photo cannot be submitted. The geotag is read from the device at capture time, not typed. Together these mean you must physically be at the problem to report it.

Two entry points share the same service: the homepage capsule (fast path) and `report-page.ts` (full form with urgency and department routing).

This also explains why HTTPS is mandatory. `getUserMedia` is blocked on plain `http`, so the app cannot be tested over a laptop's LAN address — deploying to Vercel was the only way to test on real phones.

6.2 Duplicate detection

Before a report is filed — and **before the photo uploads**, so a duplicate never puts a second copy in storage — the app calls:

```
find_duplicate_ticket(category, lat, lng)
  → nearest open report of the same category within 75m, or null
```

Distance is plain haversine (`distance_metres`), so no PostGIS extension is needed.

If a candidate is found, `duplicate-prompt.ts` shows its photo and asks *"Is this the same problem?"*

- **Yes** → adds an upvote, files nothing
- **No, mine is different** → files normally

This is a question, not a block, and that is deliberate. Proximity is a guess: two genuinely different potholes sit 40m apart on one road all the time. A rule strict enough to catch every duplicate would silence real reports, and a citizen whose valid report was silently refused has no way around it and no reason to come back. Reports with no geotag skip the check entirely, because without coordinates the only thing left to match on is the whole campus.

6.3 Upvotes and priority

Anyone can browse `/issues` by city or pincode without an account. Backing a report requires signing in, so a vote is attributable.

The database refuses, rather than the UI hiding:

- You cannot upvote your own report (manufacturing urgency)
- You cannot upvote a closed one (nothing left to prioritise)
- You cannot upvote twice (`unique (ticket_id, user_phone)`)

`upvote_count` is denormalised onto the ticket and maintained by a trigger, because PostgREST cannot `ORDER BY` an aggregate of a joined table — and ordering the desk's queue by weight is the entire point. The admin feed sorts **upvotes first, date second**, so a problem affecting twenty people stops sinking under newer reports.

6.4 The ward control desk

`/admin` — an overview strip over three tabs, each reading through ordinary RLS with the publishable key.

Tab	Source	Actions
Reports	<code>grievance_tickets</code>	filter, change status, correct category , upload resolution proof, reject with reason, CSV
Feedback	<code>platform_feedback</code>	filter by sentiment, CSV (read-only — 0003 grants SELECT only)
Citizens	<code>profiles</code>	search, per-person report counts, CSV

Two design notes:

Rejection is not in the status dropdown. It is the one transition that must carry a reason, and a dropdown cannot ask for one — so a reason-less rejection is unreachable. The citizen sees that exact reason on their profile in place of the progress trail.

Category correction matters more than rejection. The common failure is a genuine report with the wrong label — garbage filed as a pothole. Re-filing it keeps a real complaint that rejecting would throw away.

6.5 Pulse Points

The reward system, and the part most exposed to abuse.

Earning: 5 points when a ward desk moves a **campus** report out of `Submitted`. Filing earns nothing. City reports earn nothing. Rejected reports earn nothing.

Paying on *validation* rather than *submission* is the single most important anti-abuse decision in the system. If points landed on upload, the photo filter would have to be perfect — and no filter is. Because they land on human approval, junk earns nothing no matter what slips through.

Campus-only is enforced server-side. `ward_location` is free text, so a tampered client could label a city pothole "IIT Delhi". A `campuses` table holds the 25 valid names and the trigger checks against it.

Maintenance note: `campuses` must be kept in sync with `INSTITUTIONS` in `institutions.ts`. Add a campus there and not to the table and its reports silently earn nothing.

A ledger, not a balance column. Every award and spend is an append-only row; the balance is their sum. A partial unique index on `ticket_id` means one report can only ever pay out once, even if it is rejected and reopened repeatedly.

Spending is one atomic function. `redeem_reward()` checks the balance, claims a voucher code with `FOR UPDATE SKIP LOCKED`, writes the redemption and writes the negative ledger row — all in one transaction under `pg_advisory_xact_lock(hashtext(phone))`, so two taps fired together cannot both spend the same 50 points.

Rewards cost 50 or 100 points. If the code pool is empty the redemption still records as `Pending`, so the points are spent and the debt to the citizen is on the books rather than silently lost.

6.6 Escalation

Every unresolved ticket has a 24-hour clock from `last_escalated_at`, falling back to `created_at`. `escalation.ts` mirrors the server-side rule so `/escalations` can show a live countdown per ticket.

Status: the countdown UI is built; the Edge Function that sends the email is not deployed.

`0002_escalation_cron.sql` documents the `pg_cron` schedule and the exact query it must run.

7. Frontend structure

Services — `src/app/core/`

File	Lines	Role
<code>auth.service.ts</code>	244	Register, sign in, change password, admin lookup, <code>normalisePhone</code>
<code>tickets.service.ts</code>	259	File, list, public feed, duplicates, upvotes, admin workflow
<code>admin.service.ts</code>	125	Feedback, citizens, cross-table overview counts
<code>points.service.ts</code>	82	Balance, ledger, rewards, redemption
<code>media.service.ts</code>	63	Cloudinary with automatic Supabase Storage fallback
<code>feedback.service.ts</code>	40	Platform feedback, works signed out
<code>lightbox.service.ts</code>	31	Which photo is open full screen
<code>models.ts</code>	266	Every shared type and constant
<code>escalation.ts</code>	98	Client mirror of the 24-hour rule
<code>institutions.ts</code> / <code>places.ts</code>	60 / 192	26 campuses; campus + city pickers
<code>supabase.client.ts</code> / <code>supabase.errors.ts</code>	14 / 33	Client singleton; error humanising

`supabase.errors.ts` is small but load-bearing: it turns opaque Postgres codes into sentences that name the fix — `PGRST205` becomes *"The database tables do not exist yet. Run 0001_init.sql."*

Pages — `src/app/features/`

All lazy-loaded via `app.routes.ts`; `/report`, `/profile`, `/pulse-points`, `/security` and `/escalations` sit behind `authGuard`, and `/admin` behind `adminGuard`.

`home-page.ts` is the largest file at 1,542 lines — it carries the hero, the map backdrop, the campus/city picker, the category picker, the capture flow and the draft composer.

Shared — `src/app/shared/`

`camera-capture` (live camera + geotag wizard) · `photo-lightbox` + `photo-zoom` (full-screen viewer, mounted once, opened by one attribute on any `<img>`) · `duplicate-prompt` · `map-backdrop` + `map-canvas` · `feedback-launcher`.

8. Database functions and triggers

Function	Kind	What it guarantees
<code>current_phone()</code>	stable	The JWT phone claim — the identity every policy compares
<code>is_admin()</code>	definer	Admin lookup that does not recurse inside a policy
<code>set_ticket_number()</code>	trigger	<code>CP-YYYY-NNNNN</code> , never client-chosen
<code>guard_workflow_columns()</code>	trigger, definer	Citizens cannot write status, proof or rejection reason
<code>guard_profile_phone()</code>	trigger	A display name may change; the phone never can — it is the identity every policy compares
<code>guard_new_ticket()</code>	trigger, definer	Every citizen report starts at <code>Submitted</code>
<code>touch_updated_at()</code>	trigger	<code>updated_at</code> moves only on a real status change
<code>sync_profile_from_auth()</code>	trigger, definer	Mirrors identity into <code>profiles</code>
<code>award_pulse_points()</code>	trigger, definer	5 points, once per ticket, campus only, on approval
<code>pulse_balance()</code>	stable, definer	Sum of the ledger for the caller
<code>redeem_reward()</code>	definer	Atomic spend under a per-user advisory lock
<code>guard_upvote()</code>	trigger, definer	Not your own, not closed, forces <code>user_phone</code>
<code>sync_upvote_count()</code>	trigger, definer	Keeps the denormalised count exact
<code>find_duplicate_ticket()</code>	stable, definer	Nearest open match within 75m
<code>distance_metres()</code>	immutable	Haversine — no PostGIS dependency

Migrations, in order

File	Adds
<code>0001_init.sql</code>	Tables, RLS, helpers, ticket numbering, storage bucket
<code>0002_escalation_cron.sql</code>	Documented <code>pg_cron</code> schedule (function not yet deployed)
<code>0003_feedback.sql</code>	<code>platform_feedback</code> — anyone inserts, admins read
<code>0004_profiles.sql</code>	<code>profiles</code> mirror + sync trigger + backfill
<code>0005_grant_admin.sql</code>	Promotes a registered account to the desk
<code>0006_storage_no_listing.sql</code>	Security fix — stops anonymous bucket enumeration
<code>0007_reject_tickets.sql</code>	<code>Rejected</code> status, reasons, insert guard
<code>0008_pulse_points.sql</code>	Campuses, ledger, rewards, code pool, redemption
<code>0009_upvotes.sql</code>	Upvotes, public feed view, duplicate detection

Every file is idempotent and safe to re-run.

9. Build, test, deploy

```
npm install
npm start          # dev server on :4200
npm run build      # → dist/civicpulse/browser
npm test           # 13 tests, 3 files
```

Deployment is `git push`. Vercel rebuilds from `main` automatically. `vercel.json` sets the build command, the output directory, and one rewrite that sends unmatched paths to `index.html` — without it a refresh on `/admin` would 404, since no such file exists on disk. Node is pinned to `22.x` in `package.json` so the platform cannot drift onto a version Angular rejects.

Migrations are not automated. Each `.sql` file is run by hand in the Supabase SQL Editor. Deploying the frontend does not touch the database.

10. Known gaps

Honest list of what is not finished:

1. **No password recovery.** The synthetic email domain cannot receive mail, so a citizen who forgets their password has no self-service route and no admin screen exists to reset one. This is the most likely thing to hurt real users.
2. **Escalation emails are not sent.** The countdown and the documented rule exist; the Edge Function does not.
3. **The voucher pool is empty.** Redemptions record as `Pending` until real codes are stocked in `reward_codes`.
4. `campuses` and `INSTITUTIONS` are synced by hand.
5. **Photo content is not validated.** Nothing checks that a photo labelled "pothole" shows one. The rejection workflow is the human answer; automated screening is designed but deliberately unbuilt because it costs money per report.

6. **One database for dev and production.** Test data and real data share a Supabase project.